Automatic and optimized firewall reconfiguration

F. Pizzato, D. Bringhenti, R. Sisto, F. Valenza

NOMS, 7th May 2024

Politecnico di Torino
1859
netgroup
DAUIN

Network Configuration Updates

Updates in modern networks are (often) **frequent** and **small**.

- <1% affected forwarding rules and ACL rules [1]

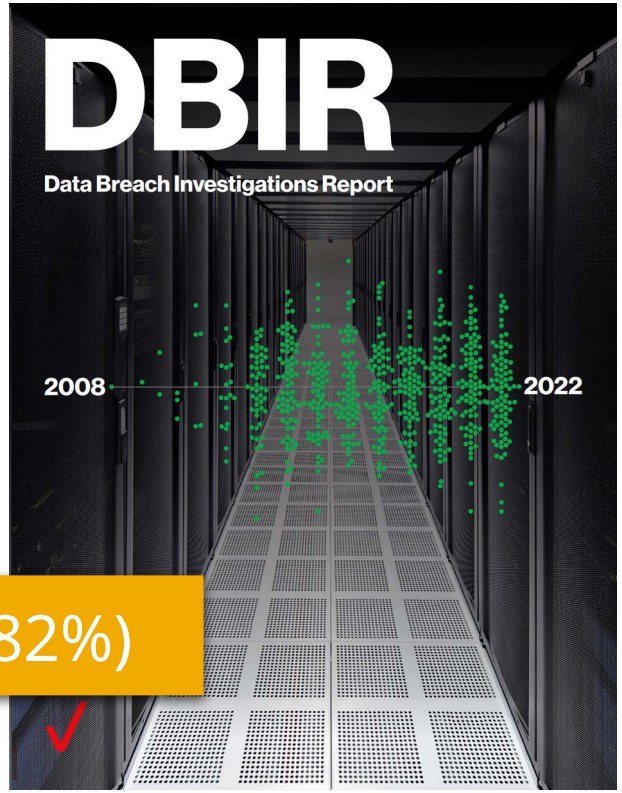
- 157 configuration lines on Facebook's network with a frequency of 12 changes/week (per device!)[2]

[1] D. Plonka and A. J. Tack, "*An Analysis of Network Configuration Artifacts*", Proc. of 23rd Large Installation System Administration Conference, USENIX Association, 2009
[2] Y.-W. E. Sung et al., "*Robotron: Top-down network management at Facebook scale*," in Proc. of the 2016 ACM SIGCOMM Conference, ser. SIGCOMM '16. New York, NY, 2016

Network Security Automation

Network Security Automation [3] is **necessary** to:

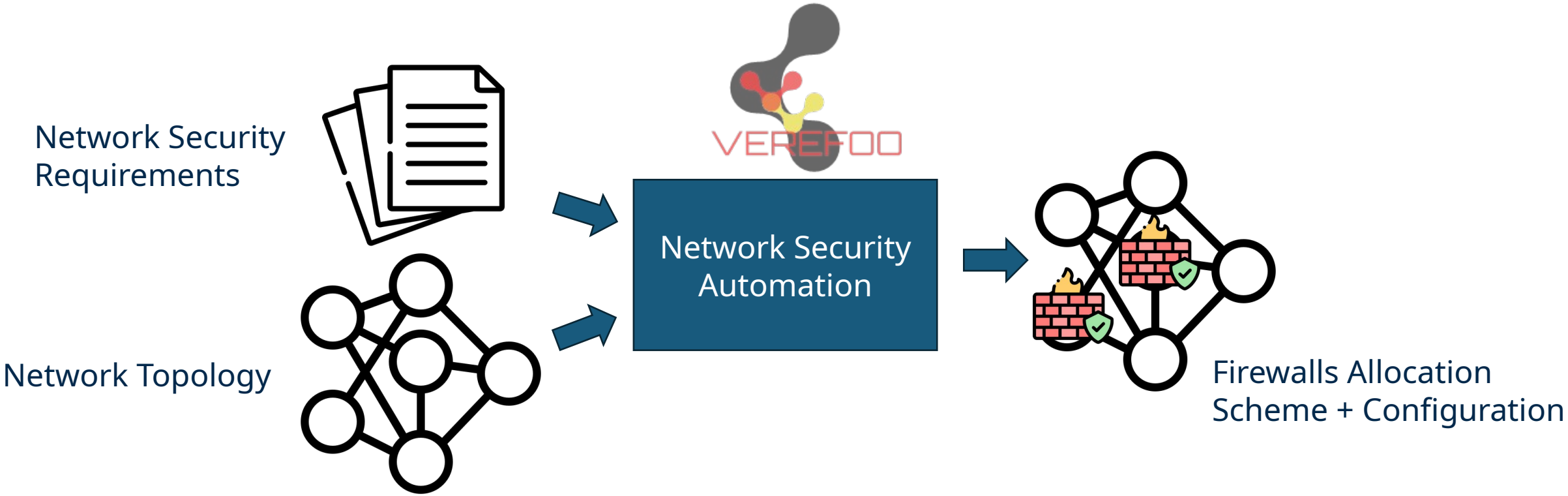
- Effectively manage complex networks
- Avoid trial-and-error and human involvement
- Additional features:
  - *Policy-based management*
  - *Formal verification*
  - *Optimization*



Human error (82%)

[3] D. Bringhenti, G. Marchetto, R. Sisto, and F. Valenza, "*Automation for network security configuration: State of the art and research trends*," ACM Comput. Surv., vol. 56, no. 3, pp. 57:1–57:37, 2024

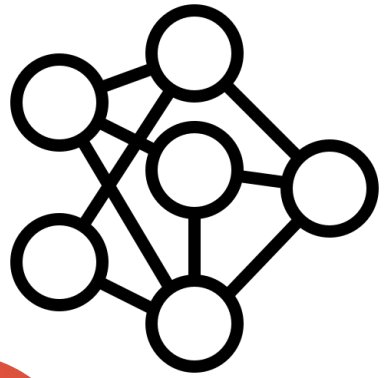
The Existing Approach



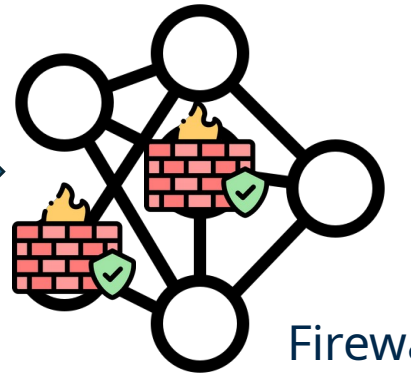
The Existing Approach

UPDATED
Network Security Requirements

Network Topology



Network Security Automation



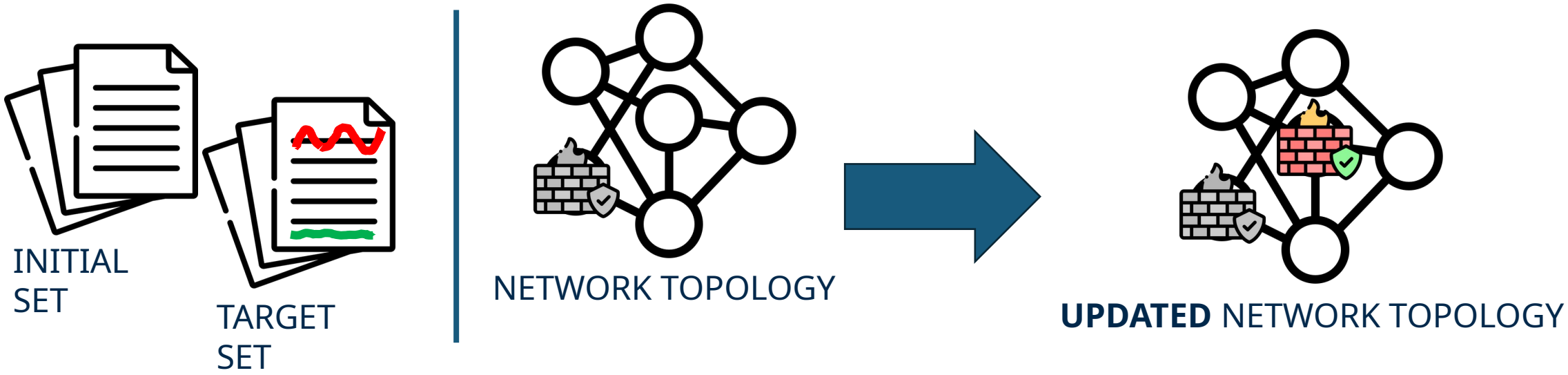
Firewalls Allocation Scheme + Configuration

When policies are updated (e.g., to mitigate an attack) the configuration of whole network must be recomputed **from scratch**

Contributions

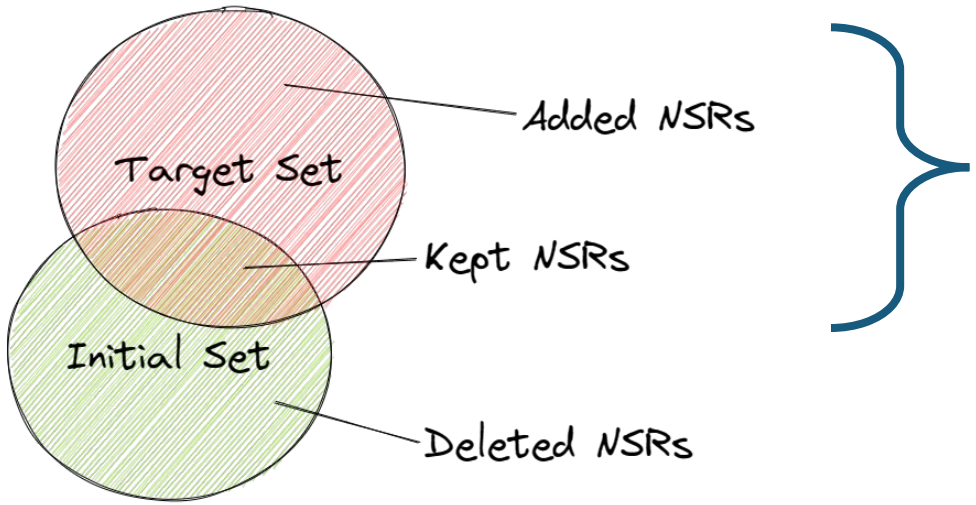
Automated approach for the *optimal* **reconfiguration** of distributed firewalls

- within a short computation time
- (formally) satisfying a **new set of Network Security Requirements** (NSRs)



Problem Inputs

- **Two sets of NSRs** + (partially configured) **Network Topology**



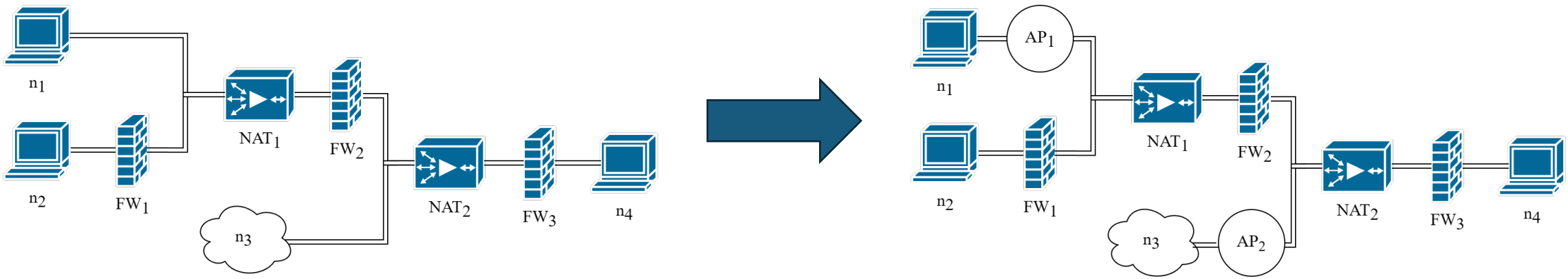
- Elements to reconfigure are selected starting from the *Added NSRs* (either new requirements or updated ones).
- Problem models *Added NSRs + Kepts NSRs* to ensure their satisfaction within the solution.

Type	IPSrc	IPDst	pSrc	pDst	tProto
Isol	192.168.0.2	220.226.50.∗	∗	∗	∗
Isol	192.168.0.3	220.226.50.∗	∗	∗	∗
Isol	130.192.225.∗	220.226.50.3	∗	∗	∗
Reach	130.192.225.∗	220.226.50.2	∗	80	TCP
Isol	130.192.225.∗	220.226.50.2	∗	≠80	TCP
Isol	130.192.225.∗	220.226.50.2	∗	∗	UDP
Reach	220.226.50.2	130.192.225.∗	∗	∗	∗
Isol	130.192.120.∗	220.226.50.2	∗	∗	∗
Reach	130.192.120.∗	220.226.50.3	∗	110	TCP
Isol	130.192.120.∗	220.226.50.3	∗	≠110	TCP
Isol	130.192.120.∗	220.226.50.3	∗	∗	UDP
Reach	220.226.50.3	130.192.120.∗	∗	∗	∗
Isol	130.192.120.∗	130.192.225.∗	∗	∗	∗

Each NSR r is modeled as a tuple (a, C), where a is the action to be applied on packets matching with the condition predicate C

Problem Inputs

- **Two sets of NSRs** + (partially configured) **Network Topology**
  - *Service Graph*, i.e., a directed graph composed of edges and vertices that represents the current network configuration.
  - This is automatically transformed into an *Allocation Graph* by placing an **Allocation Place** (AP) on every edge not including a FW.
    - APs represent the possible positions where a FW could be allocated.

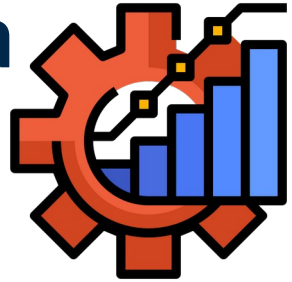


MaxSMT problem

A Maximum Satisfiability Modulo Theories (**MaxSMT**) problem differs from an SMT one because it allows the definition of two clauses:

- *Hard constraints* - compulsory
- *Soft constraints* – optional and with an associated weight

Optimization's criteria



NSRs

Hard Constraints

- Hard constraints are used to express NSRs.
- Working with packet filtering FWs, we model <u>connectivity requirements</u>:

Reachability

$$\exists f \in F_r . \forall i \left(n_i \in \pi(f) \wedge allocated(n_i) \Rightarrow \neg deny_i(\tau(f, n_i)) \right)$$

Isolation

$$\forall f \in F_r . \exists\, i . \left(n_i \in \pi(f) \wedge allocated(n_i) \wedge deny_i(\tau(f, n_i)) \right)$$

Algorithm

- Novel algorithm to select the **minimum** network area that must be reconfigured; other configuration components are kept as static elements.

- Implements a custom logic for each NSR type, i.e., Isolation or Reachability, to find all network elements to reconfigure because in conflict with at least a new NSR (i.e., *added*).

- As a result, we are **reducing the solution space** considered by the solver!

Soft Constraints

- Soft constraints are used for optimization. The two goals are:
  - *Resource minimization (i.e., number of FWs and FW rules)*
  - *Configuration state preservation*

- **Allocation** Soft Constraint

$$\forall k \mid a_k \in A_A . Soft(allocated(a_k) = false, c_k)$$

- **Configuration** Soft Constraint

$$\forall i \mid p_i \in P_k . Soft(configured(p_i) = false, c_{ki})$$

For each reconfigured FW, the previous have different weights to favor their allocation/configuration.

$$c_k^R, c_{ki}^R \mid (c_k^R < c_k) \wedge (c_{ki}^R < c_{ki})$$

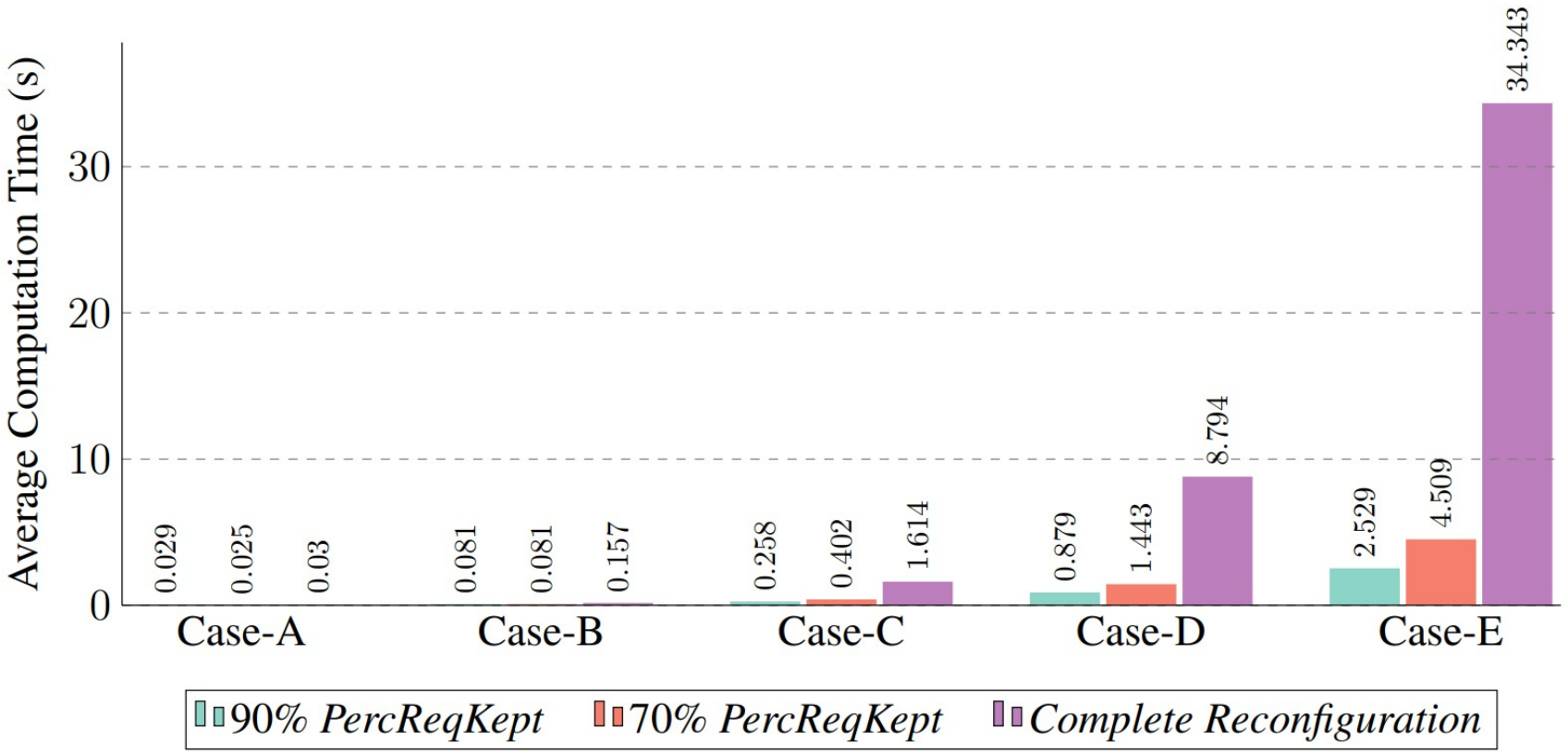
Implementation

Implementation:

- Java-based framework, with REST APIs
- z3 as MaxSMT solver

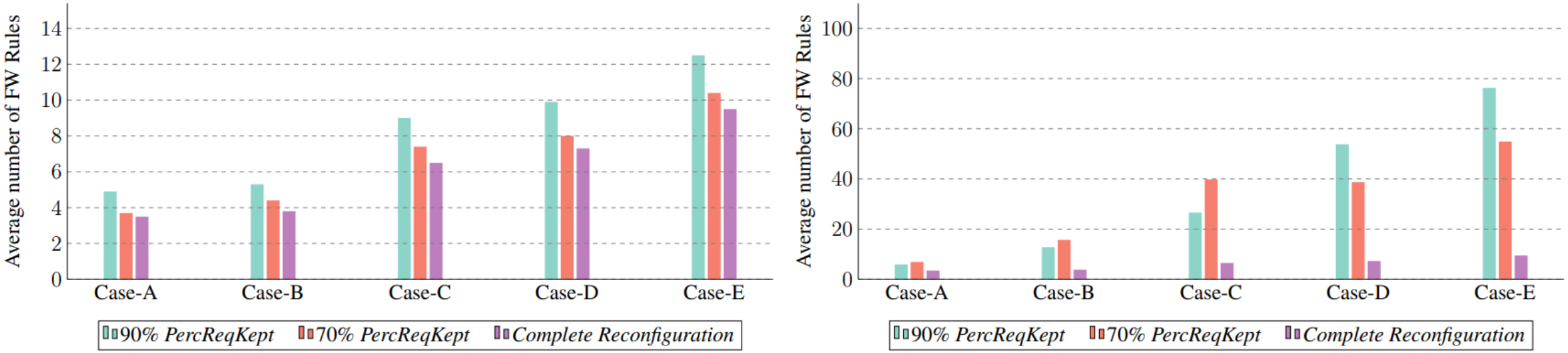
Code publicly available on GitHub (https://github.com/netgroup-polito/verefoo/tree/Dublin)

Validation - Performance



case-A: 10 NSRs, 60 endpoints, 5 NATs
case-B: 15 NSRs, 80 endpoints,10 NATs
case-C: 20 NSRs, 100 endpoints, 15 NATs
case-D: 25 NSRs, 120 endpoints, 20 NATs
case-E: 30 NSRs, 140 endpoints, 25 NATs

Validation – Optimality



(a) Results with ratio 2

(b) Results with ratio 10

- Ratio used for weights govern the balance between the two optimization objectives.
- With large value, number of used resources steeply increase to meet the goal of configuration state preservation.

Conclusions and Future Work

- A new approach has been proposed to optimize FW reconfiguration, combining **automation**, **formal verification** and **optimization**.
- Achieved better scalability with respect to the state-of-the-art in the reconfiguration scenario.
- We will try to use this work to design a parallelized approach for the resolution of the problem, dividing the network into independent areas that could be configured separately and in parallel.

Thanks for your attention!

Francesco Pizzato

<francesco.pizzato@polito.it>

GitHub: